

Điều phối lại quỹ đạo bay cho hệ thống đa UAV khi xảy ra sự cố mất điều khiển hoặc cạn pin, có xét ràng buộc tránh va chạm

Vũ Đình Nam¹

¹Khoa Cơ học kỹ thuật và Tự động hóa,
Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội
144 Xuân Thủy, Cầu Giấy, Hà Nội, Việt Nam

Abstract

Tóm tắt: Nhiệm vụ bao phủ đa UAV trên GCS thực tế thường tách lập kế hoạch tĩnh trước khi bay với failsafe từng phương tiện; khi một UAV sự cố giữa nhiệm vụ, phần residual thường bị bỏ sót hoặc được phân công lại thủ công và dễ xung đột. Khác với phân vùng serpentine offline trước chuyến bay, bài báo đề xuất lớp phục hồi *mid-mission* tập trung: phát hiện sự cố, gộp residual toàn đội, remap bằng chia nhóm kinh độ và đường lawn-mower/serpentine mới, rồi tách độ cao/thời gian khởi động (CAA) trước khi xuất .plan mid-air chuẩn QGroundControl/PX4. Kiểm chứng động học điểm trên mission thực tế cho thấy, so với không replan, remap tăng η (93,9% $\rightarrow$ 97,5%) và giảm mạnh $T_{\text{fin}}/L_{\text{max}}$; so với remap không CAA, lớp đề xuất loại N_c dự báo (1 $\rightarrow$ 0) với chi phí thời gian chỉ khoảng vài giây. Kết quả mang tính logic/động học điểm, chưa thay thế đánh giá PX4 SITL hay bay thực địa.

Từ khóa: hệ thống đa UAV; điều phối chịu lỗi; lập kế hoạch trực tuyến; bao phủ khu vực; serpentine; tránh va chạm; MAVLink; tìm kiếm cứu nạn.

1 Giới thiệu

Đội hình nhiều UAV ngày càng được sử dụng cho nhiệm vụ bao phủ trong tìm kiếm cứu nạn, đánh giá thiên tai và giám sát môi trường. Vận hành song song giúp rút ngắn thời gian hoàn thành và tăng khả năng chịu lỗi ở mức hệ thống. Tuy nhiên, một hoặc nhiều UAV có thể rời đội hình trước khi kết thúc nhiệm vụ do mất liên lạc, kích hoạt chế độ an toàn của bộ điều khiển bay, hoặc năng lượng còn lại không đủ để tiếp tục bao phủ.

Hầu hết quy trình GCS hiện nay vẫn dựa trên phân công tĩnh trước khi bay: mỗi UAV nhận một vùng và một chuỗi waypoint cố định. Khi một phương tiện sự cố, hai hệ quả không mong

muốn thường xuất hiện. Thứ nhất, phần việc còn lại của UAV hỏng bị bỏ trống nếu không có can thiệp. Thứ hai, việc phân công lại khẩn cấp dễ tạo quỹ đạo giao cắt hoặc thời điểm đến gần nhau, làm tăng rủi ro va chạm.

Bài báo tập trung vào phục hồi trực tuyến sau sự cố, không nhằm thay thế thuật toán phân vùng offline trước khi cất cánh (ví dụ [1]). Các đóng góp chính gồm:

1. Mô hình sự cố và logic phát hiện cho nhiệm vụ bao phủ đa UAV, phân biệt sự cố giữa nhiệm vụ với RTL bình thường sau khi hoàn thành nhiệm vụ.
2. Chiến lược remap residual runtime: gộp residual của UAV hỏng và các UAV sống, chia theo kinh độ, sinh lawnmower/serpentine mới (lọc centroid chỉ khi $|P| < |\mathcal{U}_a|$).
3. Lớp CAA nhẹ theo pin (tách độ cao + lệch τ), không dịch ngang hình học bao phủ sau remap.
4. Triển khai đầu cuối trên GCS MAVLink/PX4: xuất .plan mid-air, pause/RTL, nạp và khởi động lại có lịch τ .

Đóng góp không nằm ở việc cải tiến phân vùng offline kiểu [1], mà ở vòng phục hồi khi lực lượng đội bay giảm giữa nhiệm vụ.

2 Cơ sở lý thuyết và nghiên cứu liên quan

Phần này sắp xếp tài liệu theo sáu nhóm nội dung gắn trực tiếp với bài toán phục hồi quỹ đạo đa UAV khi sự cố.

Nhóm A – Lập đường bay bao phủ (CPP) và bao phủ đa UAV. Bài toán CPP đã được khảo sát rộng cho robot nói chung [2] và cho UAV nói riêng [3, 4]. Các hướng cổ điển dùng phân tách tế bào và quỹ đạo lawnmower/boustrophedon [5]. Với đa UAV, các công trình tiêu biểu phân tách đa giác rồi gán mỗi phần cho một phương tiện [6, 7], hoặc tối ưu tuyến bao phủ có nhận thức năng lượng [8]. Công trình [1] đề xuất phân vùng theo cạnh dài và định tuyến serpentine ở giai đoạn *offline trước khi bay*. Bài này cùng miền bao phủ đa UAV nhưng giải bài toán khác: khi $|\mathcal{U}|$ giảm giữa nhiệm vụ, residual được remap bằng mapper serpentine đơn giản trên GCS; bài báo **không** tái hiện hay “cải tiến” thuật toán cạnh dài của [1].

Nhóm B – Phân bổ nhiệm vụ đa robot/đa UAV. Phân loại bài toán phân bổ nhiệm vụ đa robot (MRTA) được hệ thống hóa trong [9]. Các cơ chế theo thị trường (auction/market-based) là hướng phổ biến để phân công động [10]. Thuật toán CBBA và các biến thể đồng thuận–đấu giá cho phép phân bổ phi tập trung, bền vững với bất đồng nhận thức tình huống [11, 12]. Các lược đồ phân bổ phân tán trong ngữ cảnh đa UAV cũng đã được khảo sát sớm [13].

Nhóm C – Chịu lỗi và tái phân công khi mất phương tiện. Kiến trúc chịu lỗi cổ điển như ALLIANCE minh họa khả năng robot còn lại tiếp nhận nhiệm vụ khi thành viên khác suy giảm [14]. Gần đây, các khung market-based / consensus được mở rộng cho tái lập kế hoạch khi UAV hỏng trong tìm kiếm cứu nạn hoặc đội hình suy giảm [15, 16, 17]. Điểm chung là phát hiện mất tác nhân rồi giải phóng/phân bổ lại phần việc còn lại; khác biệt của bài này nằm ở remap serpentine tập trung trên GCS kèm lớp CAA độ cao/thời gian cho mission mid-air chuẩn QGC/PX4.

Nhóm D – Tránh va chạm đa phương tiện. Velocity Obstacles [18] và Reciprocal/Optimal Reciprocal Collision Avoidance [19, 20] là nền tảng hình học cho tránh va chạm cục bộ. Với UAV, nhiều công trình kết hợp tách mặt phẳng ngang với kiểm soát/tách độ cao [21, 22]. Bài báo kế thừa tư duy tách không–thời gian nhưng áp dụng ở lớp khởi động lại nhiệm vụ (phân tầng độ cao + lệch delay), thay vì điều khiển vận tốc thời gian thực trên từng UAV.

Nhóm E – Mạng UAV, ứng dụng dân sự và cứu nạn. Khảo sát mạng UAV dân sự nhấn mạnh mất liên lạc và ràng buộc năng lượng [23]. Các tổng quan ứng dụng dân sự và thách thức then chốt (pin, va chạm, swarming) được trình bày trong [24]. Vai trò UAV trong quản lý thiên tai và SAR được phân tích trong [25, 26]. Các bài toán lập đường bay đa vùng ưu tiên cứu hộ cũng ngày càng được quan tâm [27].

Nhóm F – Nền tảng hệ thống bay và GCS. Hệ sinh thái PX4/MAVLink và API điều nhiệm vụ/telemetry là hạ tầng thực thi phổ biến cho GCS thực nghiệm [28, 29]. Tài liệu failsafe/return mode của PX4 [30] cho thấy failsafe bảo vệ từng phương tiện đơn lẻ, nhưng không tự sửa tính liên tục bao phủ ở cấp đội bay—khoảng trống mà vòng điều phối lại đề xuất hướng tới lấp đầy.

Tóm lại, GCS thực tế thường tách lập kế hoạch offline với xử lý failsafe runtime. Bài báo nói phát hiện sự cố, remap residual bằng serpentine và khởi động lại nhiệm vụ đã giải xung đột trong cùng một vòng trực tuyến tập trung.

3 Mô hình hệ thống và phát biểu bài toán

3.1 Mô hình đội bay và nhiệm vụ

Xét đội bay $\mathcal{U} = \{u_1, \dots, u_N\}$ thực hiện nhiệm vụ bao phủ dưới trạm điều khiển tập trung. Mỗi UAV u_i tại thời điểm t có vị trí $p_i(t) = (\varphi_i, \lambda_i)$, độ cao tương đối $h_i(t)$, phần trăm pin còn lại $b_i(t) \in [0, 100]$, trạng thái liên lạc $\ell_i(t) \in \{0, 1\}$, và danh sách waypoint có thứ tự $\pi_i = (w_{i,1}, \dots, w_{i,n_i})$.

Gọi $k_i(t)$ là chỉ số waypoint chưa hoàn thành gần nhất ước lượng từ $p_i(t)$. Tập waypoint còn lại của UAV i là:

$$R_i(t) = \{w_{i,k_i(t)}, \dots, w_{i,n_i}\}. \quad (1)$$

Khi sự cố xảy ra, tập residual toàn cục được định nghĩa:

$$P_{\text{remain}}(t_f) = R_f(t_f) \cup \left(\bigcup_{u_j \in \mathcal{U}_a} R_j(t_f) \right). \quad (2)$$

3.2 Sự kiện sự cố

UAV u_f được tuyên bố sự cố tại t_f nếu thỏa một trong các điều kiện sau trong lúc vẫn đang mang nhiệm vụ bao phủ chưa xong:

1. **Mất liên lạc / mất điều khiển (proxy):** không nhận telemetry hợp lệ quá T_{lost} , trong khi UAV đang in-mission hoặc đang bay ($h_i > 1$ m);
2. **Sự cố năng lượng:** $b_f(t_f) \leq b_{\text{min}}$;
3. **Chế độ failsafe:** mode thuộc $\{\text{RTL}, \text{LAND}, \text{FAILSAFE}, \text{RETURN}\}$ khi nhiệm vụ còn dở.

RTL bình thường sau khi hoàn thành nhiệm vụ được loại trừ nhờ yêu cầu cờ in-mission còn hiệu lực đối với sự cố pin và mode. Mỗi UAV chỉ bị latch sự cố một lần để tránh kích hoạt lặp.

3.3 Mục tiêu điều phối lại

Đặt $\mathcal{U}_a = \mathcal{U} \setminus \{u_f\}$ là tập UAV còn hoạt động (có GPS hợp lệ). GCS cần tính các kế hoạch mới $\{\pi'_j\}$ bao phủ P_{remain} và thỏa ràng buộc an toàn. Hàm mục tiêu đa tiêu chí dùng để đánh giá (không phải bộ tối ưu online):

$$J = \omega_1 T_{\text{fin}} + \omega_2 L_{\text{max}} + \omega_3 C_{\text{risk}}, \quad (3)$$

trong đó T_{fin} là thời gian hoàn thành dự kiến sau remap,

$$L_{\text{max}} = \max_{u_j \in \mathcal{U}_a} \text{Length}(\pi'_j) \quad (4)$$

phản ánh mất cân bằng tải, và C_{risk} là số xung đột cặp đôi dự báo theo tốc độ danh định. Ràng buộc an toàn yêu cầu:

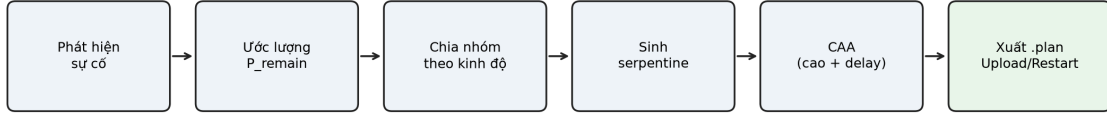
$$\|\hat{p}_i(t) - \hat{p}_j(t)\| \geq d_{\text{safe}}, \quad \forall i \neq j, \forall t \in [t_f, T_{\text{fin}}], \quad (5)$$

có thể tăng cường bằng tách độ cao $|h_i - h_j| \geq \Delta h_{\text{clear}}$ với $\Delta h_{\text{clear}} = \Delta h - 1$ (mặc định 4 m khi $\Delta h = 5$ m).

4 Phương pháp đề xuất

Quy trình đề xuất gồm năm giai đoạn thực thi tại GCS.

Quy trình điều phối lại quỹ đạo đa UAV theo sự kiện sự cố



Lớp phục hồi runtime trên GCS (bổ sung phần vùng offline)

Figure 1: Quy trình điều phối lại quỹ đạo đa UAV theo sự kiện sự cố (serpentine remap).

4.1 Giám sát runtime và phát hiện sự cố

Mỗi kênh telemetry cập nhật vector trạng thái sống $s_i = (p_i, h_i, b_i, \text{mode}_i, t_i^{\text{last}}, \text{inMission}_i)$. Việc phát hiện chạy định kỳ với chu kỳ $\Delta t \approx 1$ s.

Algorithm 1 Phát hiện sự cố

Đầu vào: trạng thái sống $\{s_i\}$, ngưỡng $T_{\text{lost}}, b_{\text{min}}$

Đầu ra: tập F các chỉ số UAV mới sự cố

```

1:  $F \leftarrow \emptyset$ 
2: for mỗi UAV  $i$  được giám sát do
3:   if  $s_i.\text{failed} = \text{true}$  then
4:     continue
5:   end if
6:    $\text{lost} \leftarrow (\text{now} - s_i.t_{\text{last}} > T_{\text{lost}})$ 
7:    $\text{lowBatt} \leftarrow (s_i.b \leq b_{\text{min}})$ 
8:    $\text{failsafe} \leftarrow (\text{mode}_i \in \{\text{RTL}, \text{LAND}, \text{FAILSAFE}, \text{RETURN}\})$ 
9:    $\text{lostFail} \leftarrow \text{lost} \wedge (\text{inMission}_i \vee h_i > 1)$ 
10:   $\text{energyModeFail} \leftarrow \text{inMission}_i \wedge (\text{lowBatt} \vee \text{failsafe})$ 
11:  if  $\text{lostFail} \vee \text{energyModeFail}$  then
12:    đánh dấu  $i$  failed;  $F \leftarrow F \cup \{i\}$ 
13:  end if
14: end for
15: return  $F$ 

```

4.2 Ước lượng residual toàn cục

Waypoint còn lại của mỗi UAV được lấy từ tập .plan đang dùng và vị trí GNSS hiện tại (chỉ số waypoint gần nhất). Cách này giảm phụ thuộc vào một luồng mission-progress đơn lẻ. Tập P_{remain} được tạo bằng cách gộp residual của UAV sự cố và mọi UAV còn sống, rồi khử trùng gần theo cửa sổ cục bộ: mỗi điểm mới chỉ được giữ nếu cách mọi điểm trong 12 điểm vừa thêm hơn 1 m (tránh $O(K^2)$ trên residual dày).

4.3 Remap vùng còn lại bằng chia nhóm kinh độ và serpentine

Đây là khác biệt chính so với chỉ “chia lại danh sách waypoint cũ”. Quy trình remap gồm:

1. Ước lượng khoảng cách lần δ từ median khoảng cách lân cận của P_{remain} (kẹp trong $[5, 40]$ m), rồi tăng δ nếu dự báo số waypoint vượt ngân sách nạp mission (khoảng 250 điểm/UAV);
2. Đặt $M' = \min(|\mathcal{U}_a|, |P_{\text{remain}}|)$. Nếu $|\mathcal{U}_a| > |P_{\text{remain}}|$ thì chỉ giữ M' UAV gần centroid(P_{remain}) nhất (tránh dải rộng); ngược lại dùng toàn bộ $\mathcal{U}_a$ có GPS hợp lệ;
3. Sắp xếp P_{remain} theo kinh độ và cắt thành M' nhóm cân bằng về số điểm;
4. Với mỗi nhóm, lập bounding box, đệm pad = $\max(0.5; 0.1 \cdot \delta)$ mét, rồi sinh đường lawn-mower/serpentine với bước δ ; điểm bắt đầu ưu tiên gần $p_j(t_f)$;
5. Gán dải theo thứ tự tây $\rightarrow$ đông cho các UAV đã chọn, sắp theo kinh độ hiện tại.

$$\pi'_j = \text{Serpentine}(\text{Pad}(\text{BBox}(G_j)), \delta, p_j(t_f)). \quad (6)$$

Algorithm 2 Serpentine remap residual

Đầu vào: chỉ số sự cố f , tập sóng $\mathcal{U}_a$ (có GPS), residual $\{R_i\}$, khoảng δ

Đầu ra: các quỹ đạo $\{\pi'_j\}$

- 1: $P \leftarrow \text{UniqueWindow}(R_f \cup (\cup_{j \in \mathcal{U}_a} R_j), \text{cửa sổ}=12, \text{ngưỡng}=1 \text{ m})$
 - 2: **if** $|P| < 2$ **then**
 - 3: **return** $\emptyset$
 - 4: **end if**
 - 5: $M' \leftarrow \min(|\mathcal{U}_a|, |P|)$; $\delta \leftarrow \text{AdaptSpacing}(\text{EstimateSpacing}(P), M', \text{ngân sách} \approx 250 \text{ WP/UAV})$
 - 6: $U' \leftarrow \mathcal{U}_a$
 - 7: **if** $|\mathcal{U}_a| > |P|$ **then**
 - 8: $U' \leftarrow M'$ UAV gần centroid(P) nhất
 - 9: **end if**
 - 10: sắp U' theo kinh độ tăng dần (Tây $\rightarrow$ Đông)
 - 11: $\{G_1, \dots, G_{M'}\} \leftarrow \text{SplitByLongitude}(P, M')$
 - 12: **for** $k = 1..M'$ **do**
 - 13: $\pi'_{U'[k]} \leftarrow \text{SerpentineInBBox}(\text{Pad}(\text{BBox}(G_k), \text{pad} = \max(0.5, 0.1\delta)), \delta, \text{start} = p_{U'[k]})$
 - 14: **end for**
 - 15: **return** $\{\pi'_j\}$
-

4.4 Tránh va chạm bằng tách độ cao và lệch thời gian

Sau khi sinh serpentine, các quỹ đạo có thể chồng lấn không-thời gian, đặc biệt trên đoạn chuyển tiếp. Lớp giải xung đột ưu tiên:

1. **Phân tầng độ cao:** $h'_j = h_j^0 + \text{rank}(j) \cdot \Delta h$. Xếp hạng theo pin giảm dần: pin cao hơn $\rightarrow$ rank nhỏ hơn $\rightarrow$ băng thấp hơn và τ nhỏ hơn (ưu tiên hoàn thành sớm trong mô hình dự báo). Độ cao nền h_j^0 lấy từ alt_rel hiện tại nếu ≥ 5 m, ngược lại dùng 10 m;

2. **Lệch thời gian khởi động:** $\tau_j = \text{rank}(j) \cdot \tau_0$; nếu vẫn còn xung đột dự báo thì tăng thêm $\tau_j += 1 + \text{rank}(j)$, lặp tối đa 4 vòng.

Vị trí dự báo $\hat{p}_j(t)$ nội suy theo polyline với tốc độ danh định $v \approx 2 \text{ m/s}$. Một xung đột được đếm khi hai UAV có $|h'_i - h'_j| < (\Delta h - 1)$ và đi vào quả cầu bán kính d_{safe} trong cùng mẫu thời gian ($\Delta t_{\text{sample}} = 1 \text{ s}$). Với $\Delta h = 5 \text{ m}$, ngưỡng bỏ qua kiểm tra ngang là 4 m—đủ để các băng độ cao danh định không bị đếm xung đột. Hình học serpentine đã sinh không bị dịch ngang thêm ở bước này.

Algorithm 3 Giải xung đột

Đầu vào: $\{\pi'_j\}$, độ cao nền h^0 , tốc độ v , d_{safe} , Δh , τ_0

Đầu ra: $\{h'_j\}$, $\{\tau_j\}$

```

1: xếp hạng UAV theo pin còn lại (giảm dần)
2: clearance  $\leftarrow \max(1, \Delta h - 1)$ 
3: for mỗi UAV  $j$  do
4:    $h'_j \leftarrow h_j^0 + \text{rank}(j) \cdot \Delta h$ 
5:    $\tau_j \leftarrow \text{rank}(j) \cdot \tau_0$ 
6: end for
7: for iter = 1..4 do
8:   if CountConflicts( $\dots$ , clearance) = 0 then
9:     break
10:  end if
11:  for mỗi UAV  $j$  do
12:     $\tau_j \leftarrow \tau_j + (1 + \text{rank}(j))$ 
13:  end for
14: end for
15: return  $\{h'_j\}$ ,  $\{\tau_j\}$ 

```

4.5 Tái sinh nhiệm vụ và khởi động lại

Các quỹ đạo đã giải xung đột được xuất ra tệp plan chuẩn QGroundControl (`replan_points{i}.plan`) và ghi đè `points{i}.plan` sau khi sao lưu bản gốc lần đầu vào `mission/backup_before_replan/`. Với khởi động lại khi đang bay, mọi mục dẫn đường dùng lệnh waypoint (MAV_CMD 16), không dùng takeoff (22), rồi kết thúc bằng RTL (20).

Trình tự runtime trên GCS: tạm dừng mọi UAV helper còn liên lạc $\rightarrow$ RTL UAV sự cố nếu còn link $\rightarrow$ nạp/khởi động các UAV nhận dải mới theo lịch tuyệt đối $t_j^{\text{start}} = t_0 + \tau_j \rightarrow$ các UAV không nhận dải mới (nếu có, ví dụ khi $|P| < |\mathcal{U}_a|$) được resume mission cũ, không gán strip mới.

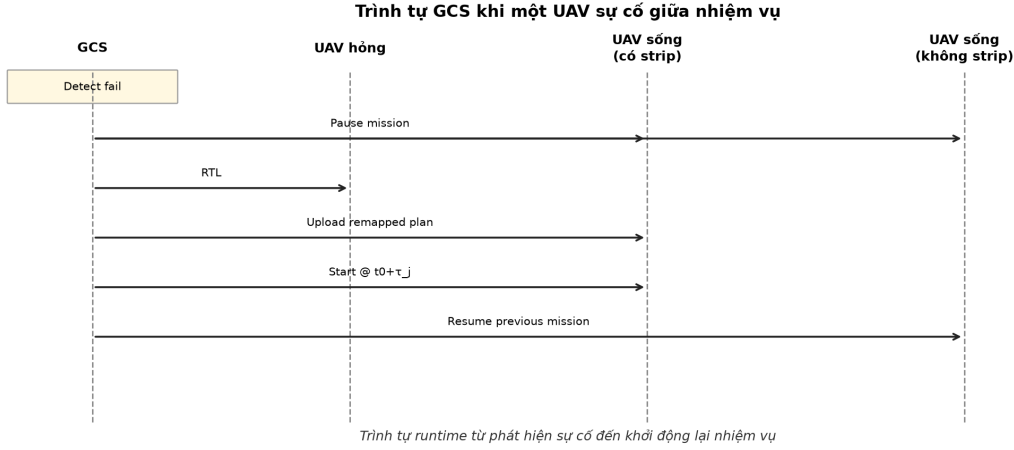


Figure 2: Trình tự runtime từ phát hiện sự cố đến khởi động lại nhiệm vụ.

5 Phân tích độ phức tạp

Gọi M là số UAV còn sống được gán dải, $K = |P_{\text{remain}}|$, T số mẫu thời gian dự báo xung đột, và L số waypoint serpentine trung bình mỗi dải. Quét sự cố $O(N)/\text{chu kỳ}$. Ước lượng spacing $O(K^2)$ trên mẫu giới hạn. Chia theo kinh độ $O(K \log K)$. Sinh serpentine $O(M \cdot L)$. Kiểm tra xung đột cặp đôi $O(M^2 T)$. Với đội bay nhỏ ($N \leq 6$) và vài trăm waypoint, vòng phục hồi phù hợp vận hành tương tác trên GCS.

Table 1: Tham số runtime mặc định (khớp `ReplanConfig` trong mã nguồn).

Tham số	Ký hiệu	Mặc định / Vai trò
Ngưỡng mất liên lạc	T_{lost}	3 s — proxy mất ĐK/liên lạc
Ngưỡng pin	b_{min}	25% — sự cố năng lượng
Khoảng cách an toàn ngang	d_{safe}	15 m — ngưỡng xung đột
Bước tách độ cao	Δh	5 m — băng theo rank pin
Ngưỡng bỏ qua xung đột đứng	$\Delta h - 1$	4 m — không đếm N_c
Bước lệch thời gian	τ_0	2 s — $\tau_j = \text{rank} \cdot \tau_0$
Số vòng tăng delay CAA	—	4 — mỗi vòng: $\tau_j += 1 + \text{rank}$
Tốc độ danh định dự báo	v	2 m/s — nội suy CountConflicts
Khoảng làn serpentine	δ	tự ước lượng (kẹp 5–40 m)
Ngân sách WP/UAV	—	~ 250 — tăng δ nếu vượt
Chu kỳ giám sát	Δt	1 s — vòng detect
Tập giám sát	—	UAV 1–5 (<code>monitor_indices</code>)
Cooldown tái điều phối	—	15 s — tránh kích hoạt dồn
Auto-replan mặc định	—	OFF — bật bằng UI / Force Replan

6 Thiết kế thực nghiệm và đánh giá

6.1 Thiết lập triển khai

Phương pháp được hiện thực thành module `replan_coordinator` tích hợp trong GCS đa UAV (Python/PyQt5, MAVSDK/qasync, QGroundControl .plan). Tập giám sát mặc định là UAV 1–5; Auto-replan mặc định tắt và có thể bật từ UI (Auto-Replan / Force Replan / Sim Fail UAV1). Harness offline `simulate_replan_experiments.py` chạy so sánh B0/B1/Proposed và xuất CSV chỉ số Bảng 3. Các kiểm thử tự động đã xác nhận: phát hiện sự cố, ước lượng residual, chia nhóm kinh độ, sinh serpentine, tách độ cao/delay, xuất tệp mid-air (cmd 16 + RTL 20), và backup trước khi ghi đè.

6.2 Các kịch bản và chiến lược so sánh

- **S1:** mất liên lạc giữa nhiệm vụ (T_{react} gồm T_{lost}).
- **S2:** sự cố pin/mode failsafe trong lúc còn in-mission (cùng tiến độ residual với S1; khác chủ yếu ở T_{react}).
- **S3:** fail sớm hơn $\rightarrow$ residual dày hơn, nhân rủi ro xung đột khi khởi động lại không CAA.

Các chiến lược:

- **B0:** không tái điều phối (UAV sống giữ residual cũ; phần UAV hỏng bỏ trống);
- **B1:** remap serpentine, không tách độ cao/thời gian;
- **Proposed:** Thuật toán 1–3 (remap + CAA).

S1 và S2 dùng chung hình học residual để tách ảnh hưởng loại sự cố lên T_{react} ; các chỉ số η , T_{fin} , L_{max} , N_c của hai kịch bản này trùng nhau như kỳ vọng. S3 không đưa B0 vì mục tiêu chính là so B1–Proposed khi residual dày.

6.3 Chỉ tiêu đánh giá

Các chỉ số được đo trên mô hình động học điểm (polyline, tốc độ danh định v), không phải footprint cảm biến thật:

$$\eta = \frac{|\{p \in P_{\text{remain}} : \exists \text{ WP trên } \cup \pi' \text{ cách } p \leq r_{\text{cov}}\}|}{|P_{\text{remain}}|}, \quad (7)$$

với $r_{\text{cov}} \approx 0,6 \cdot \delta$ (kẹp tối thiểu 8 m). Như vậy η là proxy bao phủ residual theo waypoint, không phải diện tích quét cảm biến.

$$T_{\text{fin}} = \max_j (\tau_j + \text{Length}(\pi'_j)/v), \quad L_{\text{max}} = \max_j \text{Length}(\pi'_j). \quad (8)$$

N_c = số cặp xung đột dự báo theo CountConflicts (d_{safe} , clearance = $\Delta h - 1$, $\Delta t_{\text{sample}} = 1$ s). $T_{\text{react}}(\text{S1}) = T_{\text{lost}} + t_{\text{compute}}$; với S2/S3, $T_{\text{react}} \approx t_{\text{compute}}$ (độ trễ quyết định, chưa gồm độ trễ liên lạc/điều khiển thực địa).

Table 2: So sánh định tính (đối chiếu định lượng ở Bảng 3).

Tiêu chí	B0 Không replan	B1 Remap không CAA	Đề xuất
Bao phủ residual	Thiếu phần UAV hồng (η thấp hơn)	Đường serpentine mới (η cao hơn)	Cùng hình học remap với B1
Tải / T_{fin}	L_{max} , T_{fin} cao hơn	Giảm nhờ chia lại residual	Gần B1; $+\tau$ do CAA
Xung đột restart	Thấp (ít đổi quỹ đạo)	Có thể > 0 nếu đồng mức	$N_c \rightarrow 0$ nhờ tách cao/thời gian
Can thiệp vận hành	Cần phân công thủ công	Tự remap, chưa tách an toàn	Tự remap + CAA trên GCS
Phạm vi khẳng định: offline kinematic / logic mission; chưa chứng minh an toàn bay tuyệt đối.			

Table 3: Kết quả mô phỏng động học điểm offline (mission/points1..4.plan, tiền độ fail $\approx 40\%$; $v = 2$ m/s). Chưa phải PX4 SITL/bay thật.

Kịch bản	Chiến lược	η (%)	T_{fin} (s)	L_{max} (m)	N_c	T_{react} (s)
S1	B0	93,9	1710,5	3421,1	0	3,01
S1	B1	97,5	1052,0	2104,0	1	3,04
S1	Đề xuất	97,5	1056,0	2104,0	0	3,02
S2	B0	93,9	1710,5	3421,1	0	0,01
S2	B1	97,5	1052,0	2104,0	1	0,04
S2	Đề xuất	97,5	1056,0	2104,0	0	0,02
S3	B1	100,0	1313,2	2626,5	1	0,05
S3	Đề xuất	100,0	1317,2	2626,5	0	0,03

6.4 Thảo luận

Tầng 1 – Lợi ích của remap (Proposed/B1 so với B0). Trên S1/S2, không replan để lại residual UAV hồng nên η chỉ đạt 93,9%, đồng thời $L_{\text{max}} = 3421$ m và $T_{\text{fin}} \approx 1710$ s phản ánh tải lệch trên các dải cũ. Remap residual đưa η lên 97,5% và giảm L_{max} xuống 2104 m ($T_{\text{fin}} \approx 1052$ –1056 s): lợi ích chính là phục hồi bao phủ và cân bằng tải, không phải “tối ưu toàn cục” J .

Tầng 2 – Lợi ích của CAA (Proposed so với B1). Hình học serpentine của B1 và Proposed giống nhau (η và L_{max} trùng), nhưng B1 còn $N_c = 1$ khi các dải đồng mức khởi động gần nhau. Proposed đưa $N_c \rightarrow 0$ nhờ băng độ cao và τ ; chi phí là $\Delta T_{\text{fin}} \approx 4$ s (đúng bậc $\tau_{\text{max}} = 2 \cdot \tau_0$). Trên S3 residual dày hơn, cùng mô típ: CAA loại xung đột dự báo với chi phí thời gian nhỏ, không đổi L_{max} .

Table 4: Kiểm chứng logic trên tập mission thực tế (offline, không SITL).

Hạng mục	Kết quả quan sát
Input	mission/points1..4.plan (17/16/16/270 WP)
Giả lập tiến độ	Mỗi UAV tại $\sim 40\%$ chuỗi WP (residual $\approx 12/11/11/163$)
Sự cố giả lập	UAV1 fail; alive = $\{2, 3, 4\}$
Pool residual	197 điểm sau gộp/khử trùng cửa sổ
Spacing tự ước lượng	18,2 m (không cần tăng thêm ngân sách)
Số WP sau remap	32/98/117 (tổng 247) cho UAV 2/3/4
Độ cao / delay sau CAA	$h' = 12/17/22$ m; $\tau = 0/2/4$ s (theo rank pin)
N_c trước $\rightarrow$ sau CAA	$1 \rightarrow 0$
Lệnh mid-air	Chỉ waypoint 16 + RTL 20; không có takeoff 22
Fail nối tiếp UAV2	Remap lại cho $\{3, 4\}$ thành công ($N_c: 1 \rightarrow 0$)

Tầng 3 – T_{react} . S1 bị chặn dưới bởi $T_{\text{lost}} = 3$ s; S2/S3 chủ yếu phản ánh t_{compute} (cỡ 10–50 ms trên máy thử). Không suy diễn T_{react} nhỏ thành độ trễ end-to-end thực địa.

Bảng 4 củng cố tính đúng pipeline (pool, spacing, lệnh mid-air 16+20, fail nối tiếp). Toàn bộ Bảng 3–4 là kiểm chứng logic/động học điểm trên một bộ mission; **chưa** thay thế PX4 SITL/bay thật.

Ranh giới với [1]. ICEMA giải phân vùng/serpentine offline trước chuyến bay; bài này giải phục hồi khi đội hình thay đổi giữa nhiệm vụ bằng mapper residual runtime đơn giản hơn, kèm detect và CAA khởi động lại. Hai lớp bổ sung nhau, không thay thế nhau.

7 Hạn chế và đe dọa đối với tính hợp lệ

Construct validity. Residual theo waypoint gần nhất và η theo bán kính r_{cov} chỉ là proxy; chúng không đo footprint cảm biến hay tỉ lệ diện tích địa lý thật. Bounding box từng nhóm có thể bao cả khoảng trống giữa các cụm residual.

Internal validity. Bộ dữ báo xung đột dùng polyline, v cố định, chưa mô hình gió, động học chuyển tiếp và độ trễ lệnh/upload. CAA giảm N_c trong mô hình này; không chứng minh “không va chạm” trên không phận thực.

External validity. Thí nghiệm trên $N \leq 4$ UAV và một bộ mission/points1..4.plan; GCS tập trung — nếu trạm mất thì không bảo đảm phục hồi đội bay. Mapper runtime không tái sử dụng thuật toán cạnh dài của [1], nên không so “chất lượng phân vùng offline”.

8 Kết luận

Bài báo đề xuất lớp điều phối lại quỹ đạo đa UAV theo sự kiện sự cố cho nhiệm vụ bao phủ trên GCS tập trung: detect $\rightarrow$ remap serpentine residual $\rightarrow$ CAA độ cao/thời gian $\rightarrow$ xuất mission mid-air MAVLink/PX4. Trên bộ mission thực tế (động học điểm), remap cải thiện η từ 93,9% lên 97,5% và giảm mạnh $T_{\text{fin}}/L_{\text{max}}$ so với không replan; lớp CAA loại xung đột dự báo ($N_c: 1 \rightarrow 0$) với chi phí thời gian khoảng vài giây, không đổi hình học bao phủ. Khung này bổ sung cho lập kế hoạch offline (ví dụ [1]), chứ không thay thế phân vùng trước chuyến bay. Hướng tiếp theo: lập lại trên PX4 SITL/bay thật; đưa residual polygon/DEM vào remap; gán dải có nhận thức năng lượng; và nghiên cứu phục hồi khi liên lạc GCS suy giảm.

Lời cảm ơn

Nghiên cứu liên quan định hướng ứng dụng trong đề tài cấp cơ sở mã số CN25.05 tại Trường Đại học Công nghệ – ĐHQGHN.

References

- [1] V. Đ. Nam, “Planning for multi-UAV control systems: Edge-oriented partitioning and serpentine routing,” in *Proc. ICEMA 2025*, 2025. DOI: 10.15625/vap.2026.0020.
- [2] E. Galceran and M. Carreras, “A survey on coverage path planning for robotics,” *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1258–1276, 2013.
- [3] T. M. Cabreira, L. B. Brisolara, and P. R. Ferreira Jr., “Survey on coverage path planning with unmanned aerial vehicles,” *Drones*, vol. 3, no. 1, p. 4, 2019.
- [4] G. Fevgas, T. Lagkas, V. Argyriou, and P. Sarigiannidis, “Coverage path planning methods focusing on energy efficient and cooperative strategies for unmanned aerial vehicles,” *Sensors*, vol. 22, no. 3, p. 1235, 2022.
- [5] H. Choset, “Coverage for robotics – A survey of recent results,” *Annals of Mathematics and Artificial Intelligence*, vol. 31, pp. 113–126, 2001.
- [6] I. Maza and A. Ollero, “Multiple UAV cooperative searching operation using polygon area decomposition and efficient coverage algorithms,” in *Distributed Autonomous Robotic Systems 6*, pp. 221–230, Springer, 2007.
- [7] G. S. C. Avellar, G. A. S. Pereira, L. C. A. Pimenta, and P. Iscold, “Multi-UAV routing for area coverage and remote sensing with minimum time,” *Sensors*, vol. 15, no. 11, pp. 27783–27803, 2015.

- [8] C. Di Franco and G. Buttazzo, “Energy-aware coverage path planning of UAVs,” in *Proc. IEEE Int. Conf. Auton. Robot Syst. Competitions (ICARSC)*, pp. 111–117, 2015.
- [9] B. P. Gerkey and M. J. Matarić, “A formal analysis and taxonomy of task allocation in multi-robot systems,” *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.
- [10] M. B. Dias, R. Zlot, N. Kalra, and A. Stentz, “Market-based multirobot coordination: A survey and analysis,” *Proceedings of the IEEE*, vol. 94, no. 7, pp. 1257–1270, 2006.
- [11] H.-L. Choi, L. Brunet, and J. P. How, “Consensus-based decentralized auctions for robust task allocation,” *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.
- [12] L. Brunet, H.-L. Choi, and J. P. How, “Consensus-based auction approaches for decentralized task assignment,” in *AIAA Guidance, Navigation and Control Conference*, 2008.
- [13] T. Lemaire, R. Alami, and S. Lacroix, “A distributed tasks allocation scheme in multi-UAV context,” in *Proc. IEEE ICRA*, pp. 3622–3627, 2004.
- [14] L. E. Parker, “ALLIANCE: An architecture for fault tolerant multirobot cooperation,” *IEEE Transactions on Robotics and Automation*, vol. 14, no. 2, pp. 220–240, 1998.
- [15] W. T. Barnawi, G. M. Nicholls, C. D. McAllister, and A. Y. Ortiz, “Integrated consensus framework for task assignment and path planning of a degraded UAV fleet,” *Drones and Autonomous Vehicles*, vol. 2, no. 4, p. 10016, 2025.
- [16] Y. Zeng, L. Wu, J. Li, and X. Zhuang, “Enhancing UAV swarm resilience to payload failures: A bidirectional task reallocation approach,” *IEEE Systems Journal*, 2026.
- [17] L. Giacomossi *et al.*, “Market-based replanning for safety-critical UAV swarms in search and rescue missions,” 2026. arXiv:2606.01970.
- [18] P. Fiorini and Z. Shiller, “Motion planning in dynamic environments using velocity obstacles,” *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.
- [19] J. van den Berg, M. Lin, and D. Manocha, “Reciprocal velocity obstacles for real-time multi-agent navigation,” in *Proc. IEEE ICRA*, pp. 1928–1935, 2008.
- [20] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics Research*, pp. 3–19, Springer, 2011.
- [21] E. Ferrera, A. Alcántara, J. Capitán, A. R. Castaño, P. J. Marrón, and A. Ollero, “Decentralized 3D collision avoidance for multiple UAVs in outdoor environments,” *Sensors*, vol. 18, no. 12, p. 4101, 2018.

- [22] S. Huang, R. Teo, and W. Liu, “Distributed cooperative avoidance control for multi-unmanned aerial vehicles,” *Actuators*, vol. 8, no. 1, p. 1, 2019.
- [23] S. Hayat, E. Yanmaz, and R. Muzaffar, “Survey on unmanned aerial vehicle networks for civil applications: A communications viewpoint,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 4, pp. 2624–2661, 2016.
- [24] H. Shakhathreh *et al.*, “Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges,” *IEEE Access*, vol. 7, pp. 48572–48634, 2019.
- [25] M. Erdelj, E. Natalizio, K. R. Chowdhury, and I. F. Akyildiz, “Help from the sky: Leveraging UAVs for disaster resilience,” *IEEE Pervasive Computing*, vol. 16, no. 1, pp. 24–32, 2017.
- [26] M. Erdelj and E. Natalizio, “UAV-assisted disaster management: Applications and open issues,” in *Proc. ICNC*, pp. 1–5, 2016.
- [27] L. Du, Y. Fan, M. Gui, and D. Zhao, “A multi-regional path-planning method for rescue UAVs with priority constraints,” *Drones*, vol. 7, no. 12, p. 692, 2023.
- [28] L. Meier, D. Honegger, and M. Pollefeys, “PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms,” in *Proc. IEEE ICRA*, pp. 6235–6240, 2015.
- [29] MAVSDK Development Team, “Mission and telemetry APIs.” <https://mavsdk.mavlink.io>, 2024.
- [30] PX4 Development Team, “Failsafe and return mode documentation.” <https://docs.px4.io>, 2024.